Steps to install a Virtual Machine on the system:

Oracle VM Virtual Box

Steps:

Note: On Windows make sure to enable virtualization in BIOS:

https://www.tutorialspoint.com/windows10/windows10_virtualization.htm

1) Install Oracle VM VirtualBox

Installing on Windows Hosts.

- Click on the link to download the executable https://download.virtualbox.org/virtualbox/6.1.4/VirtualBox-6.1.4-136177-Win.exe

- In addition, Windows Installer must be present on your system. This should be the case for all supported Windows platforms.
  2.1.2 Performing the Installation
- The Oracle VM VirtualBox installation can be started in either of the following ways:
- By double-clicking on the executable file.
- By entering the following command: VirtualBox---Win.exe -extract This will extract the installer into a temporary directory, along with the .MSI file.
- Using either way displays the installation Welcome dialog and enables you to choose where to install Oracle VM VirtualBox, and which components to install.

Installing on Mac OS X

Performing the Installation for Mac OS X hosts, Oracle VM VirtualBox ships in a dmg disk image file. Perform the following steps to install on a Mac OS X host:

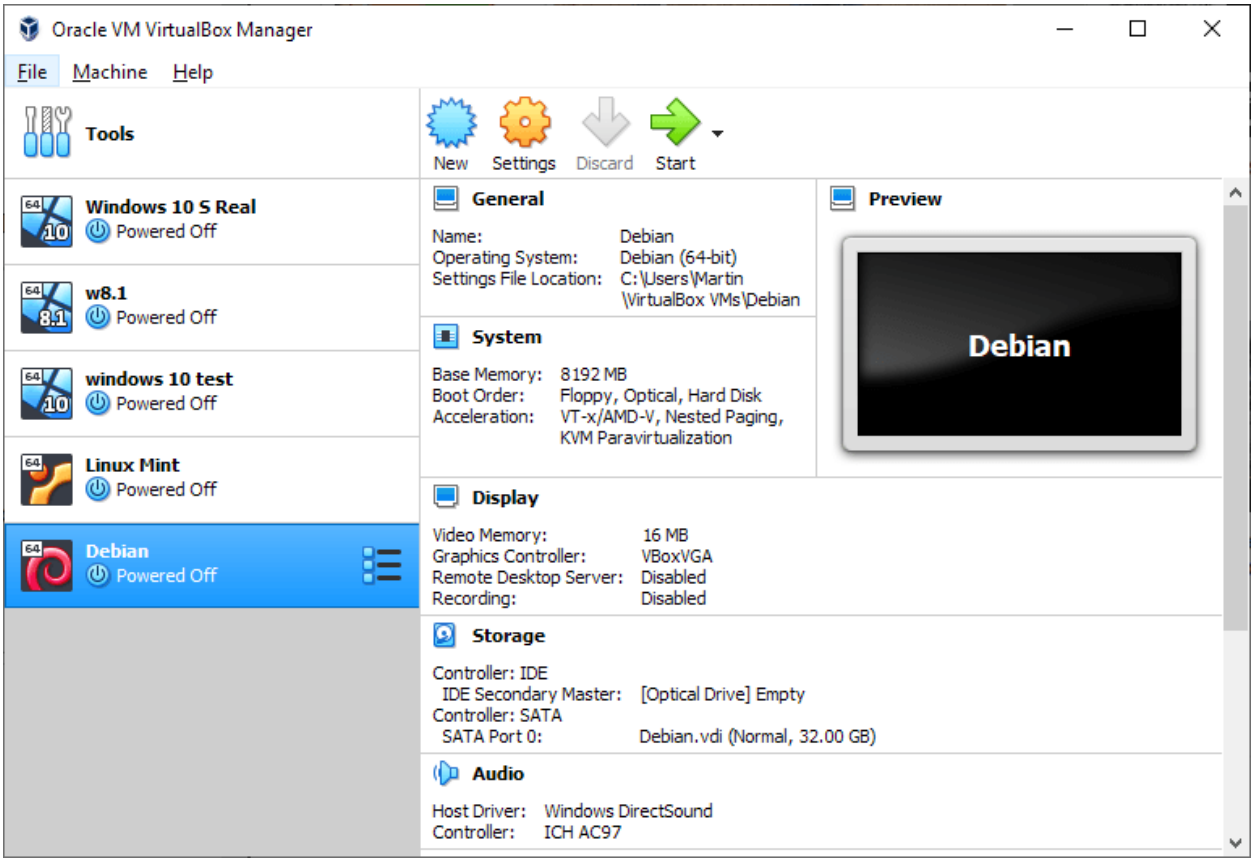
- Click on the link to download the dmg file https://download.virtualbox.org/virtualbox/6.1.4/VirtualBox-6.1.4-136177-OSX.dmg

- Double-click on the dmg file, to mount the contents.

- A window opens, prompting you to double-click on the VirtualBox.pkg installer file displayed in that window.
- This starts the installer, which enables you to select where to install Oracle VM VirtualBox.
- An Oracle VM VirtualBox icon is added to the Applications folder in the Finder.

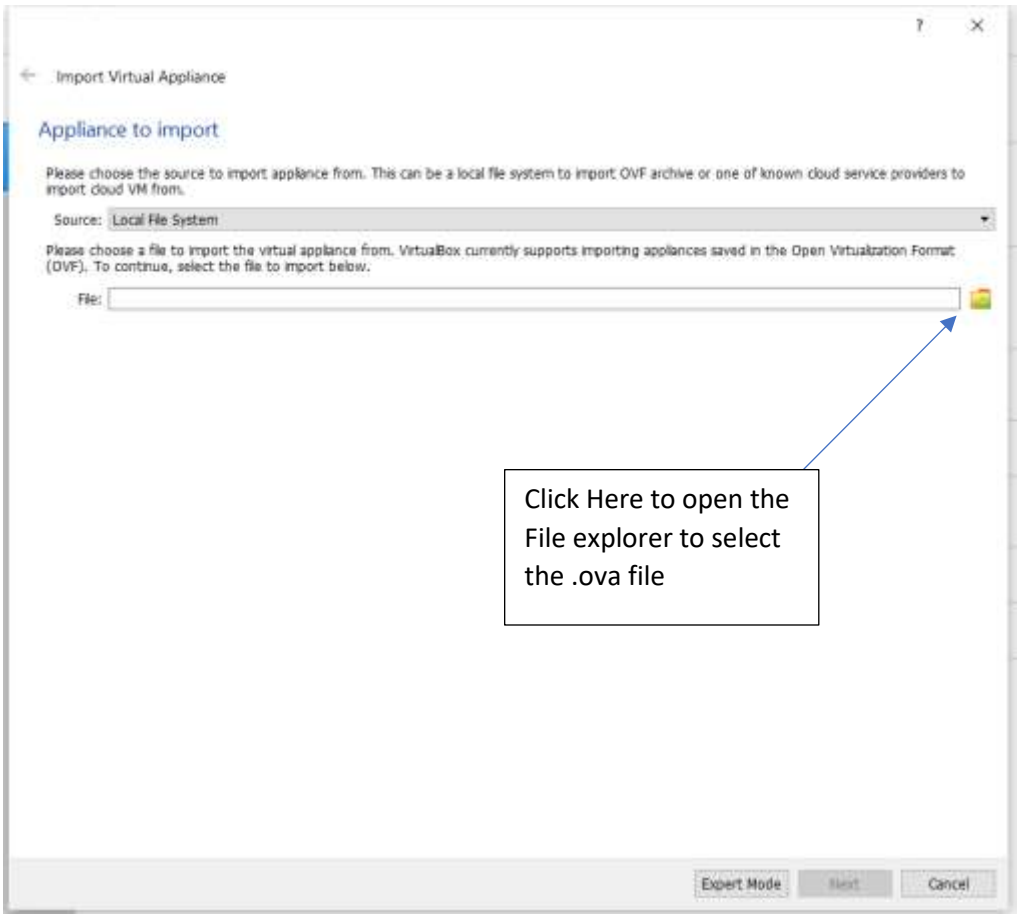
2) Install the Virtual Machine on the Virtual Box:

1) Open the Oracle VM VirtualBox

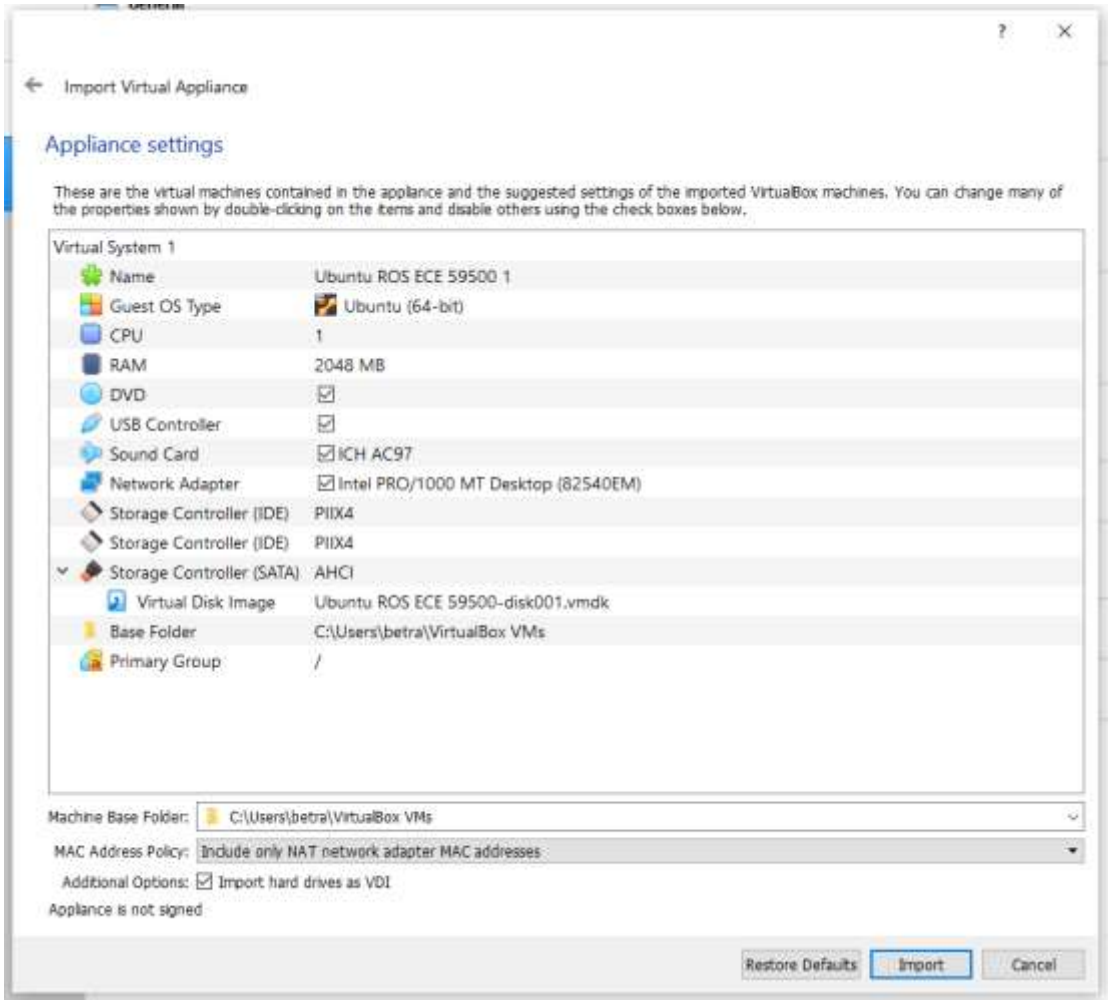


2) Click on File → Import Appliance (The following window should open)

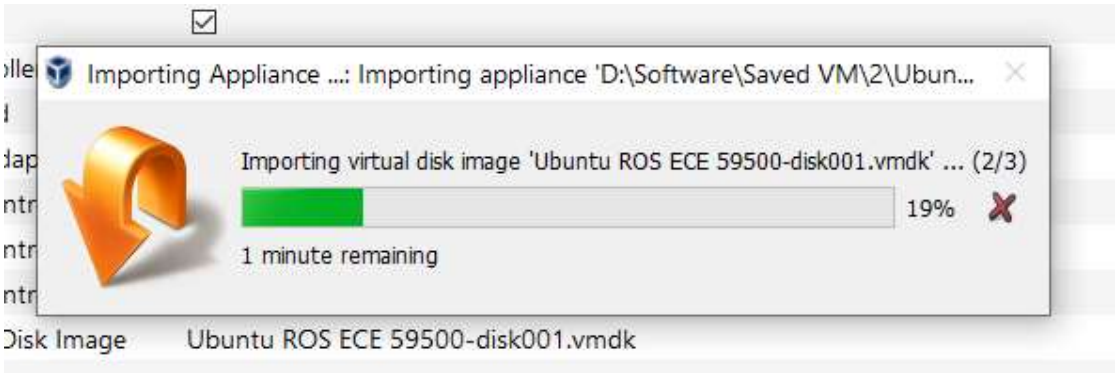
3) Click on the open folder icon as shown below.



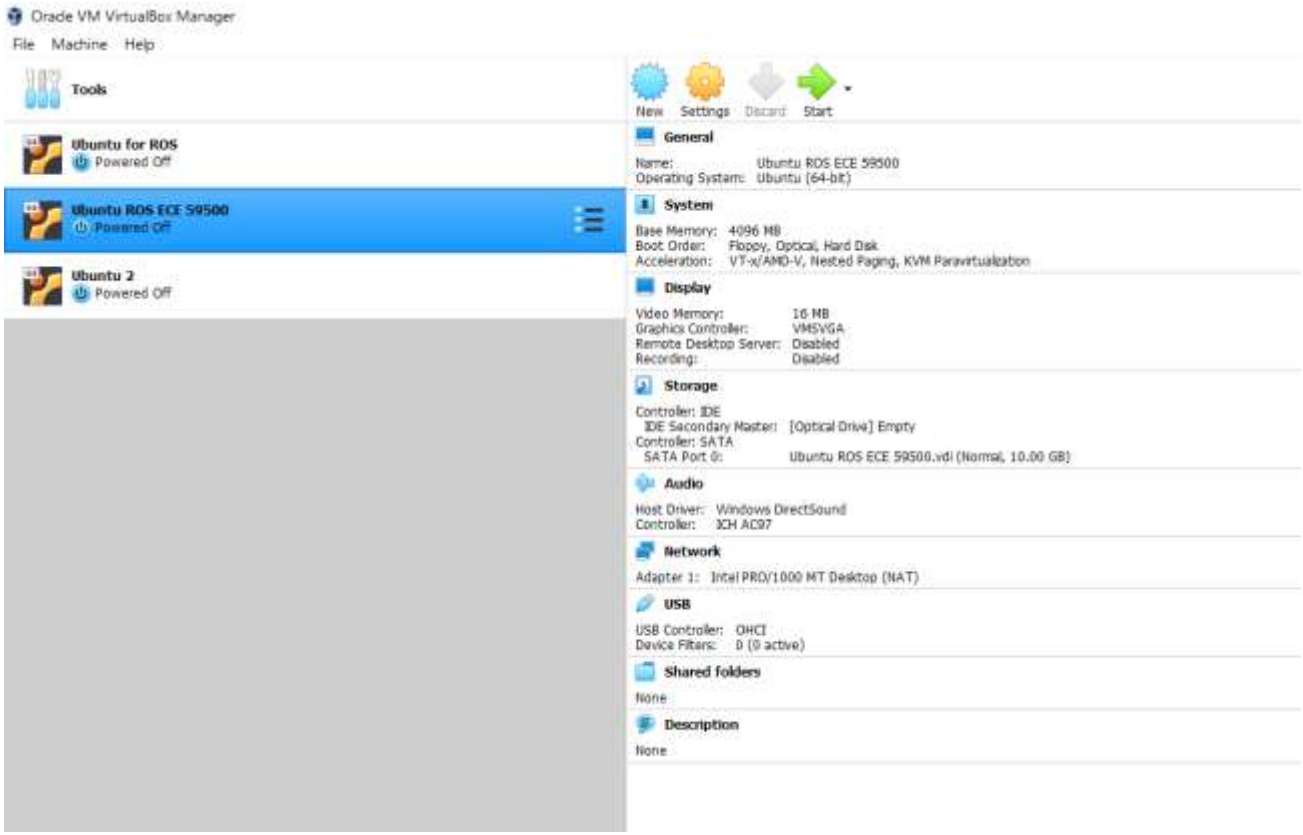
3) Click on the .ovf file provided in the problem folder.
4) Then click on Next. The dialog box below will show up. Click on Import.



5) It will start importing the Virtual Machine or “Appliance”



6) The Virtual Machine named Ubuntu ROS 56900 should then show up in the left sidebar



7) The Desktop should come up.

This Ubuntu is preinstalled with all the required ROS programs, python example files, launch files.

Username: ece

Password: ece

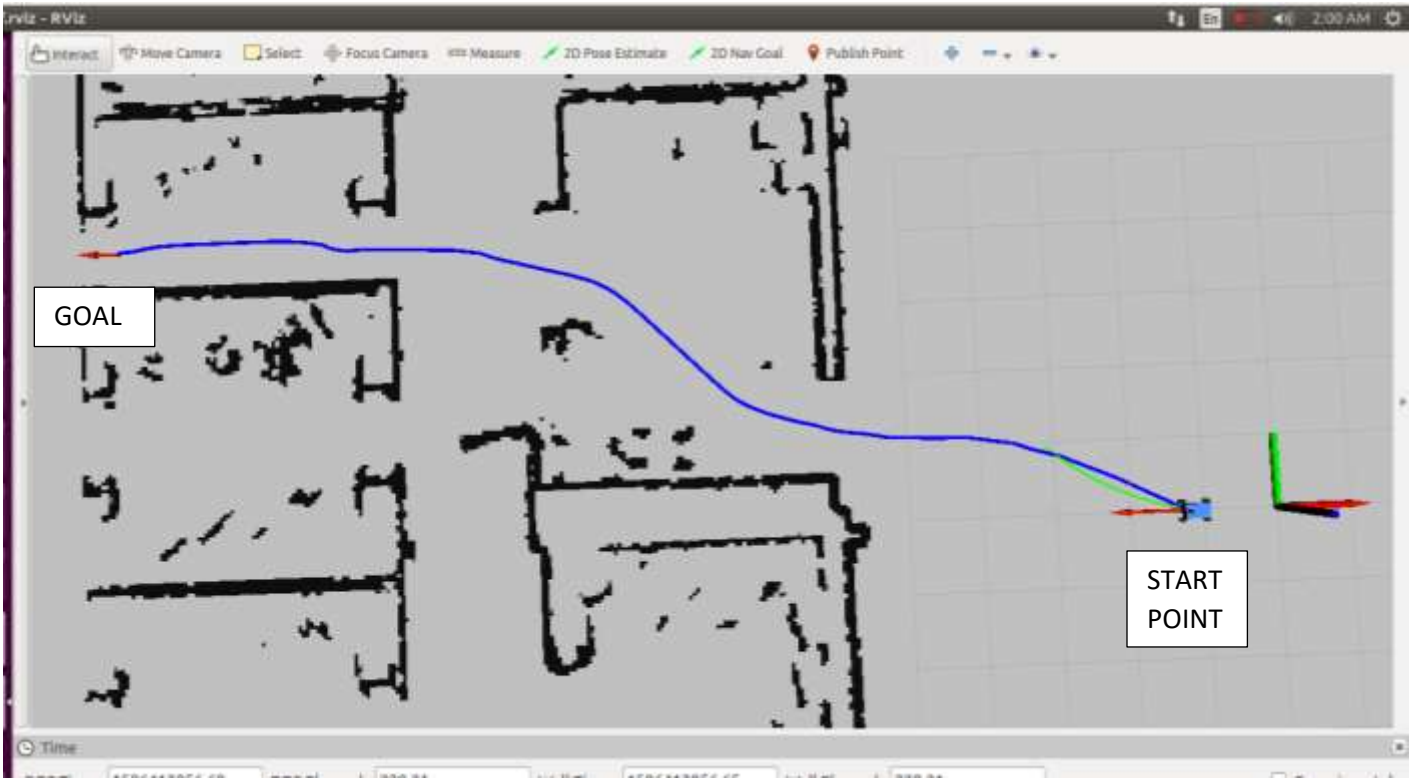
Steps to start the racecar simulator:

1) Open a terminal by either pressing Ctrl + Alt + t, or clicking on Ubuntu Home (Start)
2) In the terminal, enter the following command:
   roslaunch racecar_simulator main_simulator.launch
   This starts the ROSCORE and opens up the simulator in the rviz software.
3) The following rviz based simulator program should open. You can zoom and move the main map area using the scroll wheel and the left click. To zoom into a particular area on the map, click on focus camera button and then click on a point on the map of interest, then zoom in.

4) Open up a separate terminal in the home folder (Ctrl+ Alt+t) and enter:

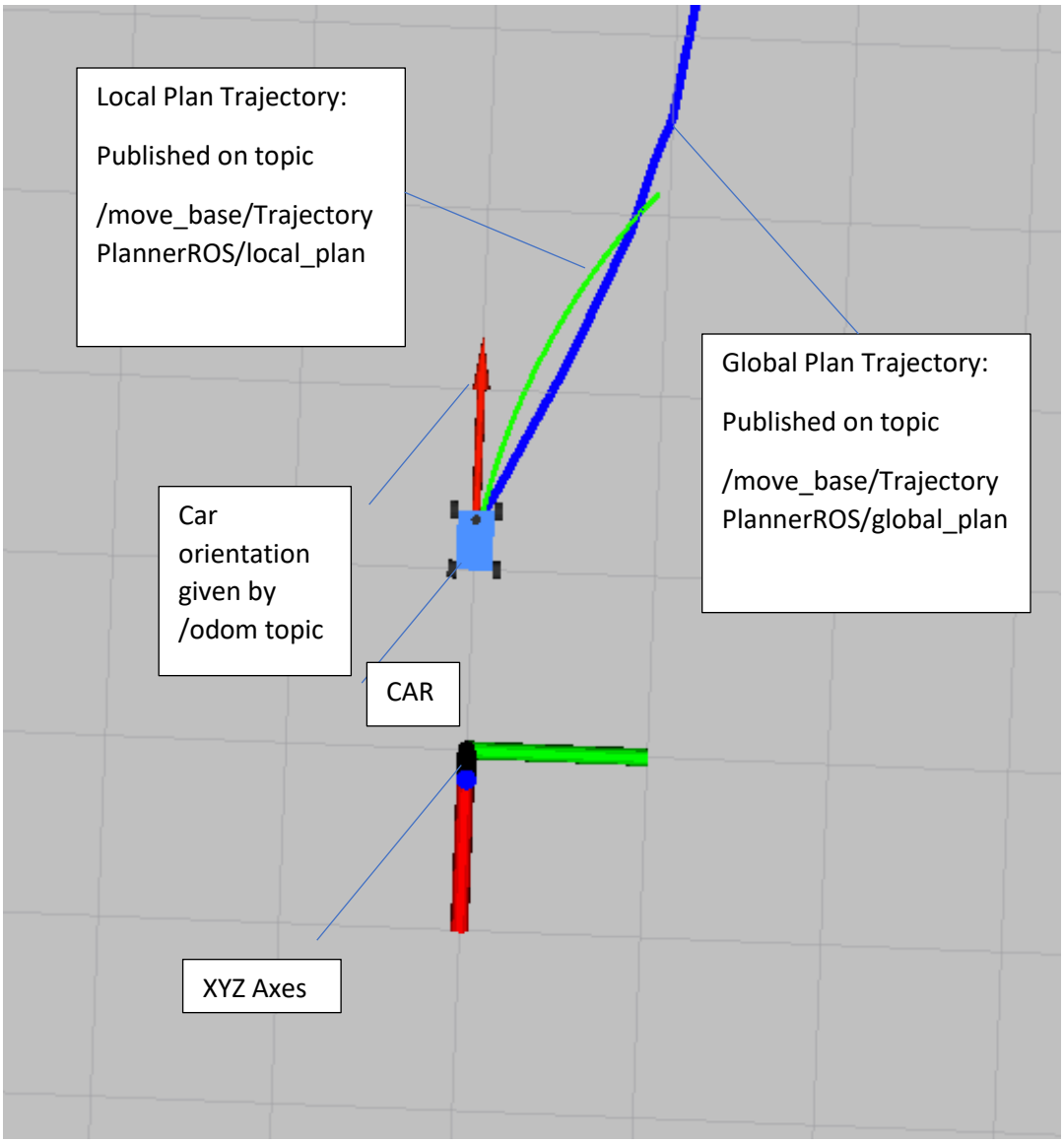
python set_pose_and_goal.py

This sets the position of the car and and the goal. The Trajectory planner initialized via the launch file automatically calculates the Local and Global Trajectories.



5) This changes the position of the car and should create and a thick blue line that represents the global trajectory as shown above and a thin green line representing the local trajectory going from the car crossing to a point on the global trajectory.

Note: This trajectory is a basic ROS trajectory planner for simple holonomic/differential drive robots. More advanced planners are required for proper Ackermann Steering based robots. A simple PID control of just the steering angle does not suffice for many trajectories due to the Ackermann drive. The objective of this problem is to give a basic overview of ROS and a bit of idea regarding its implementation.



6) The front wheels of the car and the speed of the car can be controlled using the AckermannDriveStamped() message published on the /drive topic.

7) **The main task for the student is to edit this control.py code Edit the control.py Python file by writing a small piece of code to design a PD controller and to publish Ackermann messages to drive the car to roughly follow the local trajectory (and thus the global trajectory), reach the goal and stop at the goal.**

8) Hint: The car can be moved onto the global path by using the angle difference between the Car angle and Local path to adjust the steering angle while keeping velocity constant via AckermannDriveStamped message.

9) **The file can be edited using the gedit application. You can do this by simply double clicking on the .py file as well. The code contains comments and details as to where and how the edits need to be done.**

10) Run the python file in terminal using the command

python control.py

11) Important: Make sure to always run the set_pose_and_goal.py python code to reset the car back to the starting position (as shown in image in point 11) once before running your control.py

12) An example for using Ackermann Drive is given in the ackermanndrive_example.py file. You can run it in terminal using the

python ackermanndrive_example.py

13) Feel free to use the Stackoverflow, Github or any of the following links

http://wiki.ros.org/ROS/Tutorials

http://wiki.ros.org/

14) All python files are explained in the files themselves with comments

Final Submission Requirements:

1) The final edited control.py file. You can edit the python file for submission by using the Firefox browser in the Ubuntu Virtual Machine itself.

2) A video of the Ubuntu screen with the simulation map of the car moving from start position to goal (a decent video shot from a phone is fine).